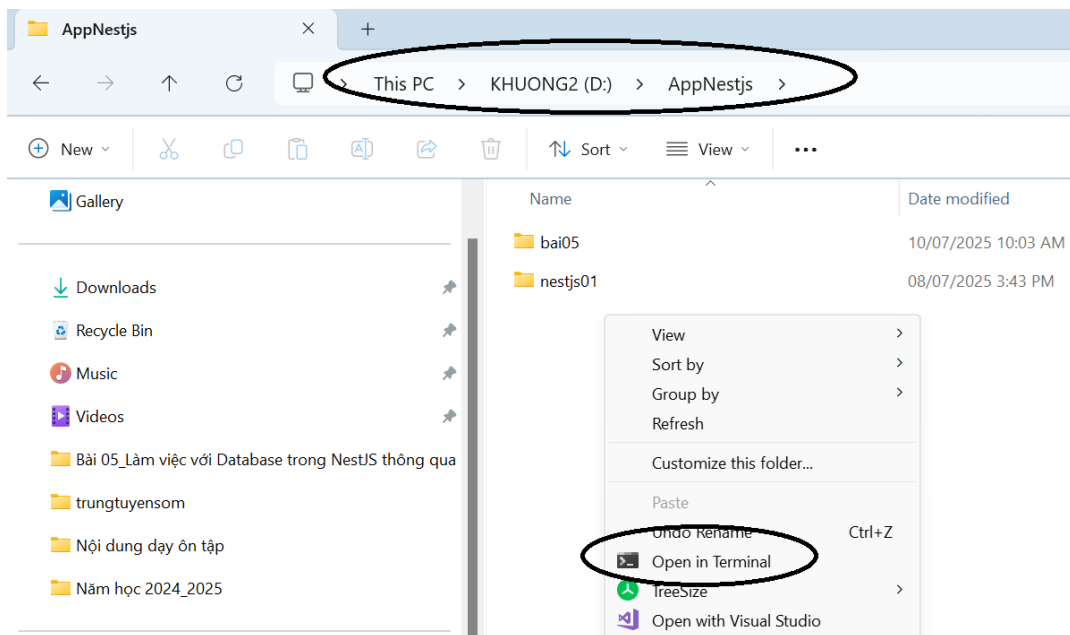


BÀI 06: XÂY DỰNG RESTFUL API CRUD HOÀN CHỈNH TRONG NESTJS

Mục tiêu bài 06: Bài này sẽ giới thiệu đến các bạn từng bước để tạo một ứng dụng hoàn chỉnh Nestjs với các chức năng Thêm, hiển thị, sửa và xóa dữ liệu (CRUD) với cơ sở dữ liệu MySQL.

6.1. TẠO DỰ ÁN NESTJS

- Đầu tiên các bạn tạo dự án **Bai06** trong thư mục **AppNestjs** đã tạo ở bài trước, bằng cách: click chuột phải vào vùng trống của thư mục đó, chọn **Open in Terminal**



- Xuất hiện giao diện **cmd**, các bạn gõ lệnh: **nest new Bai06** (để tạo dự án Bai06). Chọn **npm**, tiếp tục bấm **Enter**. Chờ cho quá trình tạo dự án diễn ra và báo quá trình tạo dự án thành công.

```
Windows PowerShell
CREATE bai06/nest-cli.json (179 bytes)
CREATE bai06/package.json (2104 bytes)
CREATE bai06/README.md (5126 bytes)
CREATE bai06/tsconfig.build.json (101 bytes)
CREATE bai06/tsconfig.json (565 bytes)
CREATE bai06/src/app.controller.ts (286 bytes)
CREATE bai06/src/app.module.ts (259 bytes)
CREATE bai06/src/app.service.ts (150 bytes)
CREATE bai06/src/main.ts (236 bytes)
CREATE bai06/src/app.controller.spec.ts (639 bytes)
CREATE bai06/test/jest-e2e.json (192 bytes)
CREATE bai06/test/app.e2e-spec.ts (699 bytes)

✔ Installation in progress...

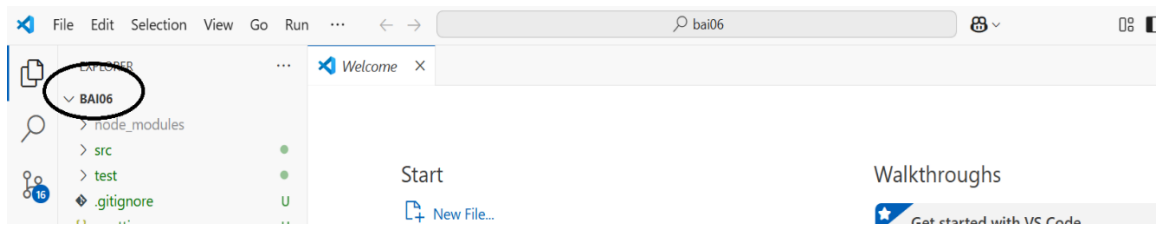
🚀 Successfully created project bai06
👉 Get started with the following commands:

$ cd bai06
$ npm run start

Thanks for installing Nest 🙌
Please consider donating to our open collective
to help us maintain this package.

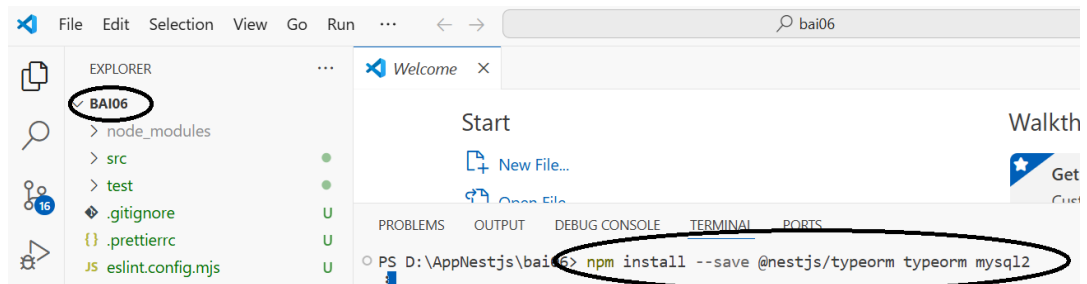
Donate: https://opencollective.com/nest
```

- Mở chương trình **VSCode** lên, mở thư mục dự án **Bai06**



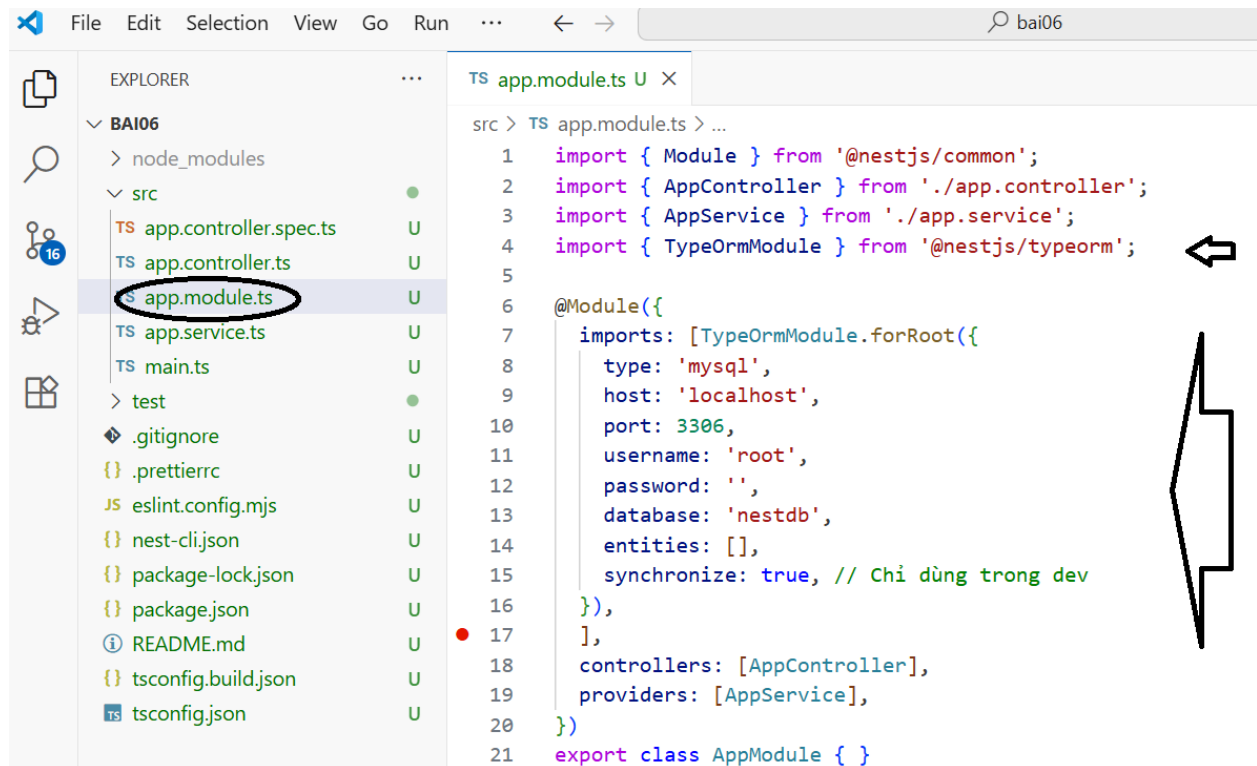
6.2. CÀI ĐẶT TYPEORM VÀO DỰ ÁN

- Các bạn mở **Terminal** của **VSCode** gõ dòng lệnh: **npm install --save @nestjs/typeorm typeorm mysql2**



6.3. CẤU HÌNH KẾT NỐI TYPEORM

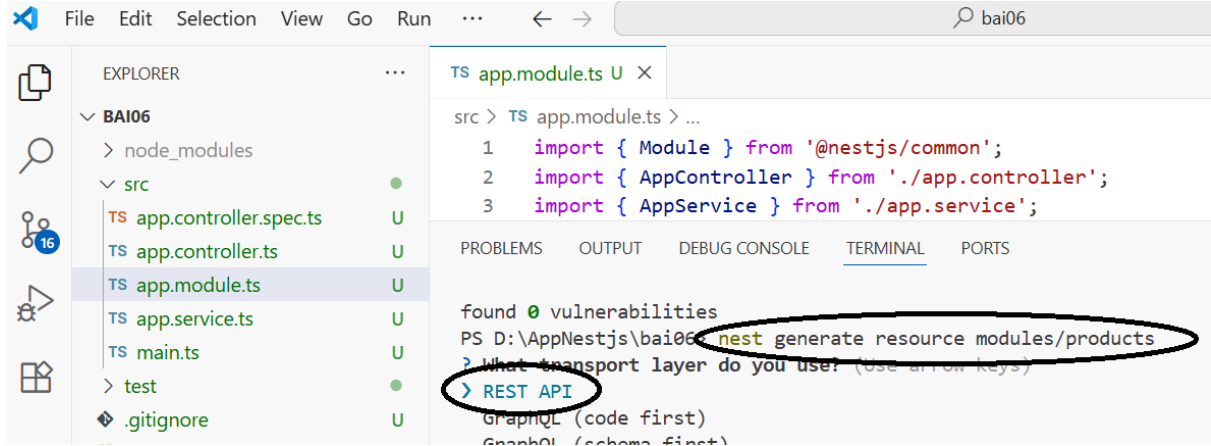
- Các bạn mở file **app.module.ts** và viết code như bên dưới:



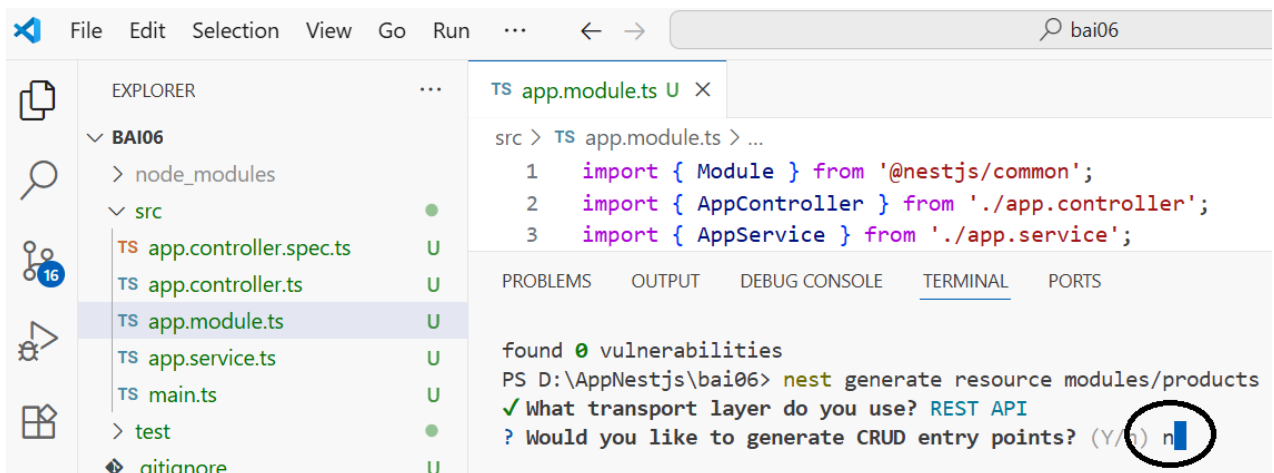
- Các bạn có thể sử dụng lại cơ sở dữ liệu **nestdb**, sẽ tạo **bảng dữ liệu khác** để lưu trữ cho bài học này.

6.4. CÀI ĐẶT MODULE VÀ ENTITY

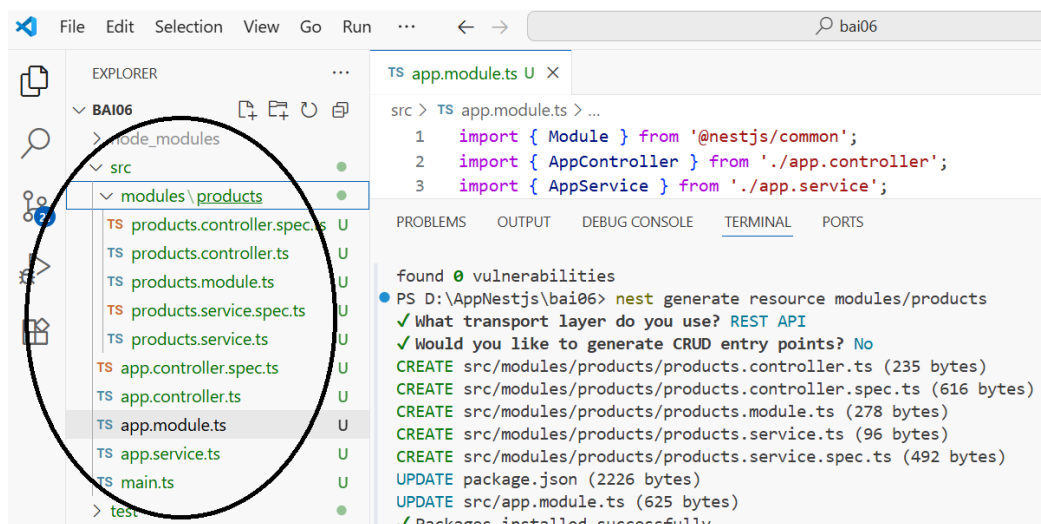
- Bây giờ chúng ta sẽ tạo **module Product** cho ứng dụng bằng cách mở **Terminal** của **VSCode** và thực hiện dòng lệnh: **nest generate resource modules/products**. Xuất hiện giao diện bên dưới, các bạn chọn **REST API** và bấm **Enter**.



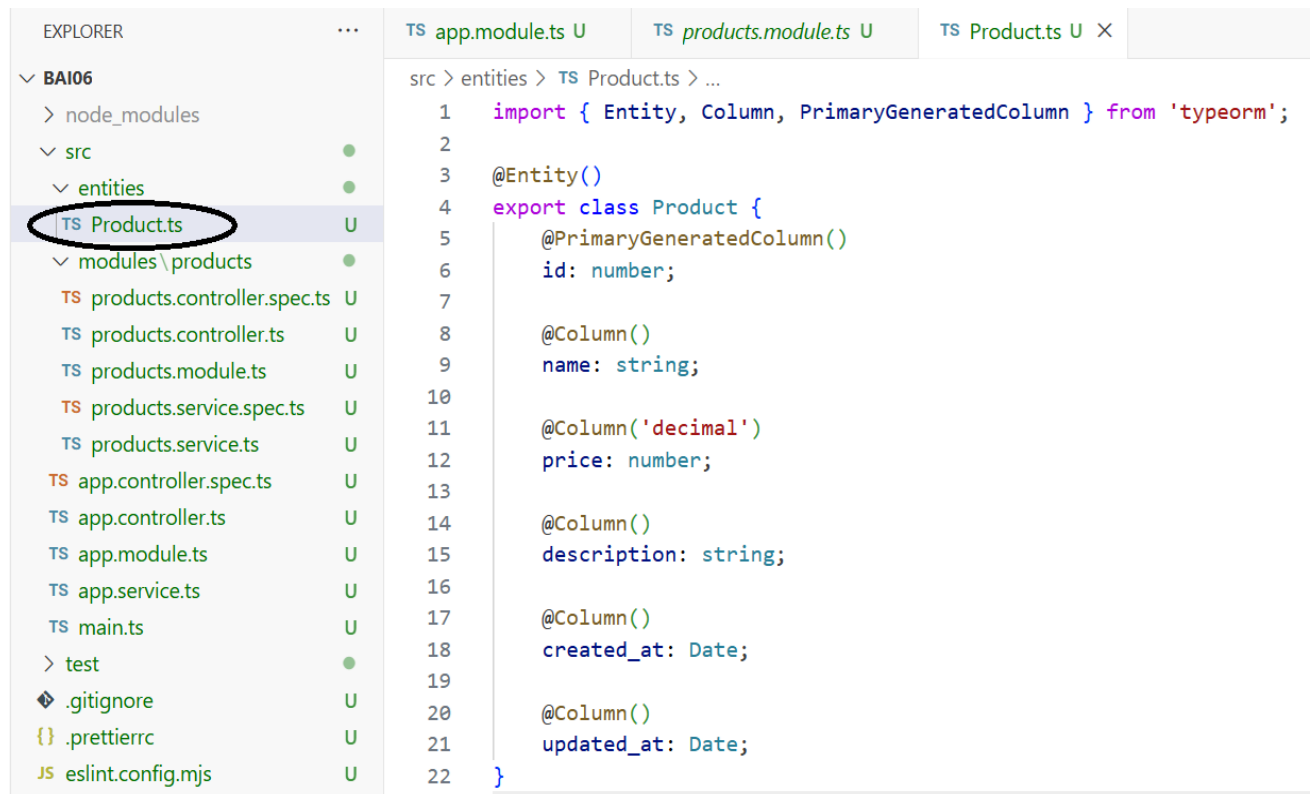
- Xuất hiện giao diện tiếp theo bên dưới, các bạn gõ: **n** và bấm **Enter**.



- Chờ một lát cho quá trình cài đặt xong, sẽ tự động tạo các **module** cho dự án chúng ta.



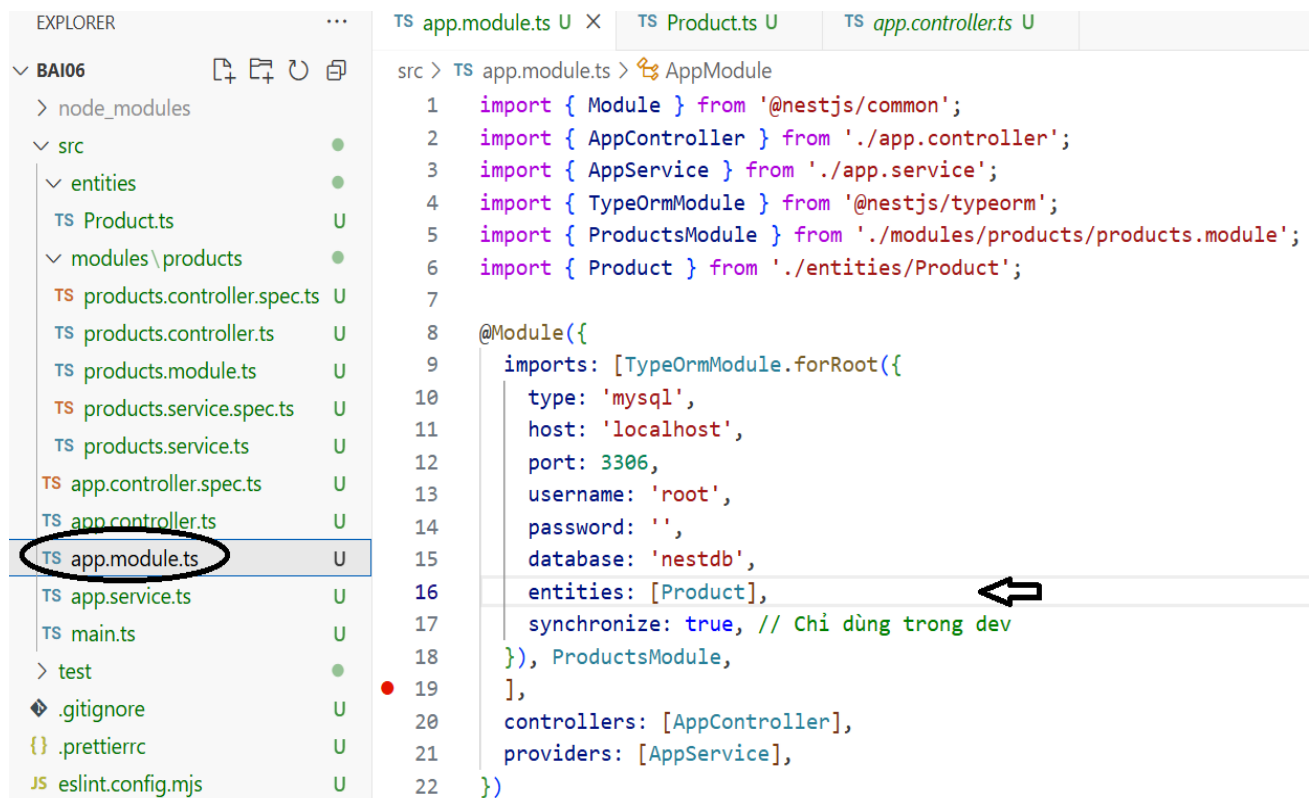
- Bây giờ chúng ta tiếp tục tạo các **Entity** bằng cách: Tạo một thư mục **entities** trong thư mục **src**. Tiếp tục tạo trong thư mục **entities** tập tin **Product.ts** và viết code như bên dưới:



```

src > entities > TS Product.ts > ...
1  import { Entity, Column, PrimaryGeneratedColumn } from 'typeorm';
2
3  @Entity()
4  export class Product {
5      @PrimaryGeneratedColumn()
6      id: number;
7
8      @Column()
9      name: string;
10
11     @Column('decimal')
12     price: number;
13
14     @Column()
15     description: string;
16
17     @Column()
18     created_at: Date;
19
20     @Column()
21     updated_at: Date;
22 }
    
```

- Tiếp tục gọi **Entity** vừa tạo vào **Module** ánh xạ đến cơ sở dữ liệu bằng cách: Mở tập tin **app.module.ts** và viết thêm code như sau:



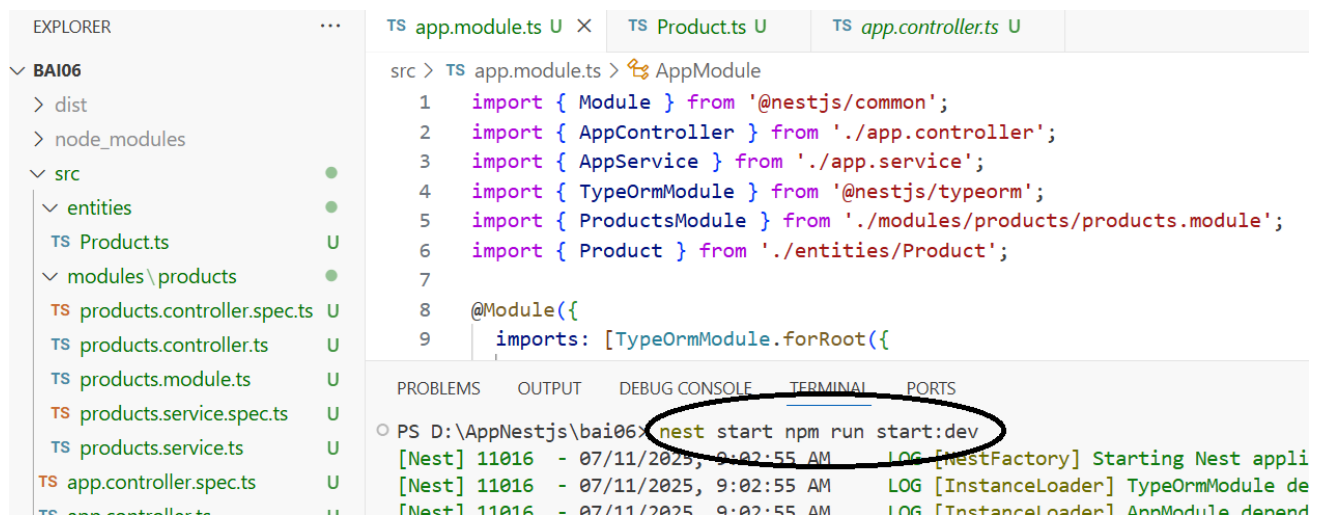
```

src > TS app.module.ts > AppModule
1  import { Module } from '@nestjs/common';
2  import { AppController } from './app.controller';
3  import { AppService } from './app.service';
4  import { TypeOrmModule } from '@nestjs/typeorm';
5  import { ProductsModule } from './modules/products/products.module';
6  import { Product } from './entities/Product';
7
8  @Module({
9      imports: [TypeOrmModule.forRoot({
10         type: 'mysql',
11         host: 'localhost',
12         port: 3306,
13         username: 'root',
14         password: '',
15         database: 'nestdb',
16         entities: [Product], // Chỉ dùng trong dev
17         synchronize: true,
18     })], ProductsModule,
19     ],
20     controllers: [AppController],
21     providers: [AppService],
22 })
    
```

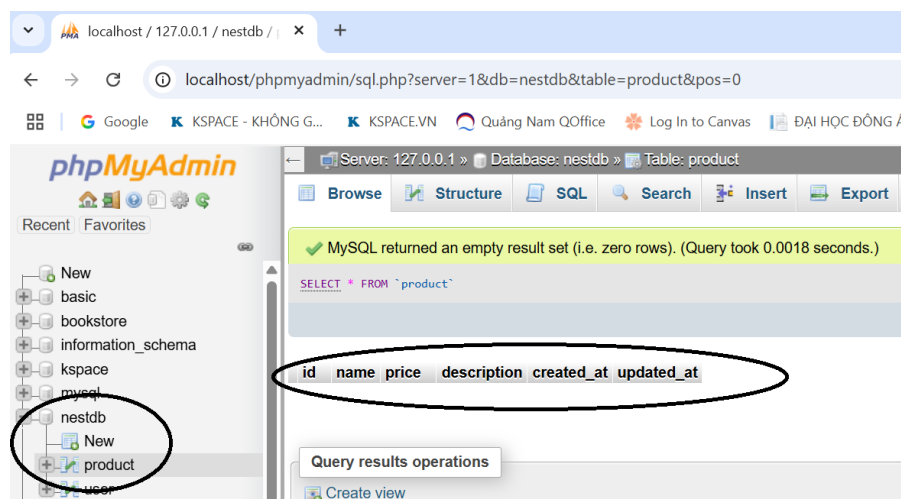
- Tiếp tục, các bạn mở tập tin **products.module.ts** và viết thêm code như sau:



- Các bạn mở **Terminal** và biên dịch chương trình với dòng lệnh: **nest start npm run start:dev**



- Bây giờ các bạn mở cơ sở dữ liệu ra sẽ thấy có thêm một bảng **Product** trong cơ sở dữ liệu **nestdb** (Hiện tại vẫn chưa có dữ liệu gì)



6.5. TẠO SERVICE THỰC HIỆN CRUD

- Để tạo Service thực hiện các thao tác trên cơ sở dữ liệu các bạn mở tập tin **products.service.ts** và viết lệnh như bên dưới:



```

1  import { Injectable } from '@nestjs/common';
2  import { InjectRepository } from '@nestjs/typeorm';
3  import { Product } from 'src/entities/Product';
4  import { Repository } from 'typeorm';
5  @Injectable()
6  export class ProductsService {
7      constructor(@InjectRepository(Product) private readonly productsRepository: Repository<Product>,) { }
8      create(productData: Partial<Product>) {
9          const product = this.productsRepository.create(productData);
10         product.created_at = new Date();
11         product.updated_at = new Date();
12         return this.productsRepository.save(product);
13     }
14     findAll() {
15         return this.productsRepository.find();
16     }
17     findOne(id: number) {
18         return this.productsRepository.findOneBy({ id });
19     }
20     async update(id: number, productData: any) {
21         await this.productsRepository.update(id, productData);
22         return this.productsRepository.findOneBy({ id });
23     }
24     async delete(id: number) {
25         const product = await this.productsRepository.findOneBy({id});
26         await this.productsRepository.delete(id);
27         return product;
28     }
29 }
    
```

6.6. ĐƯA CÁC SERVICE VÀO CONTROLLER

- Tiếp tục để đưa các Service đã tạo vào Controller để sử dụng các bạn mở tập tin **products.controller.ts** và viết lệnh như bên dưới:

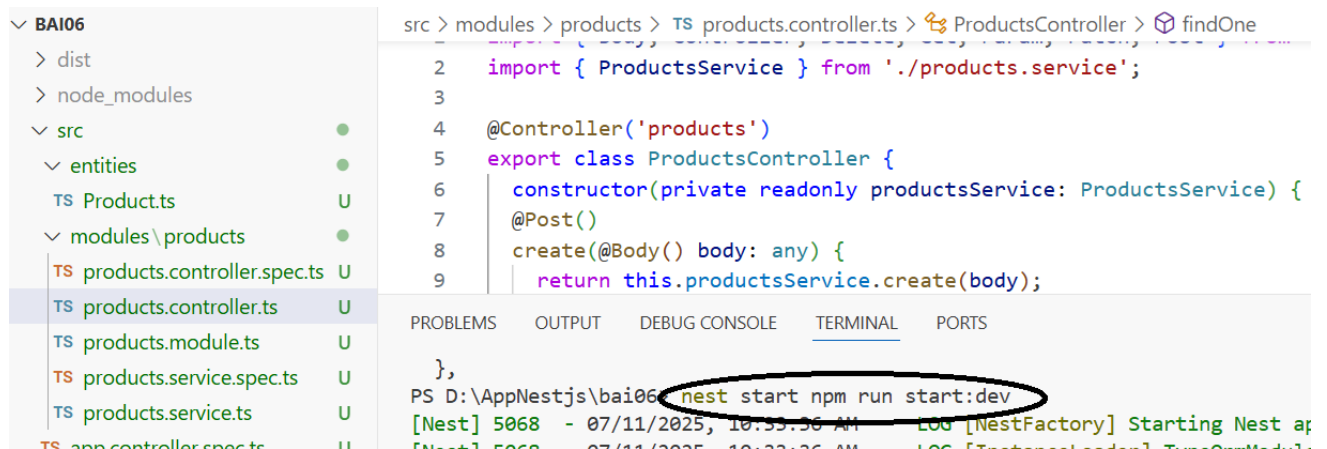


```

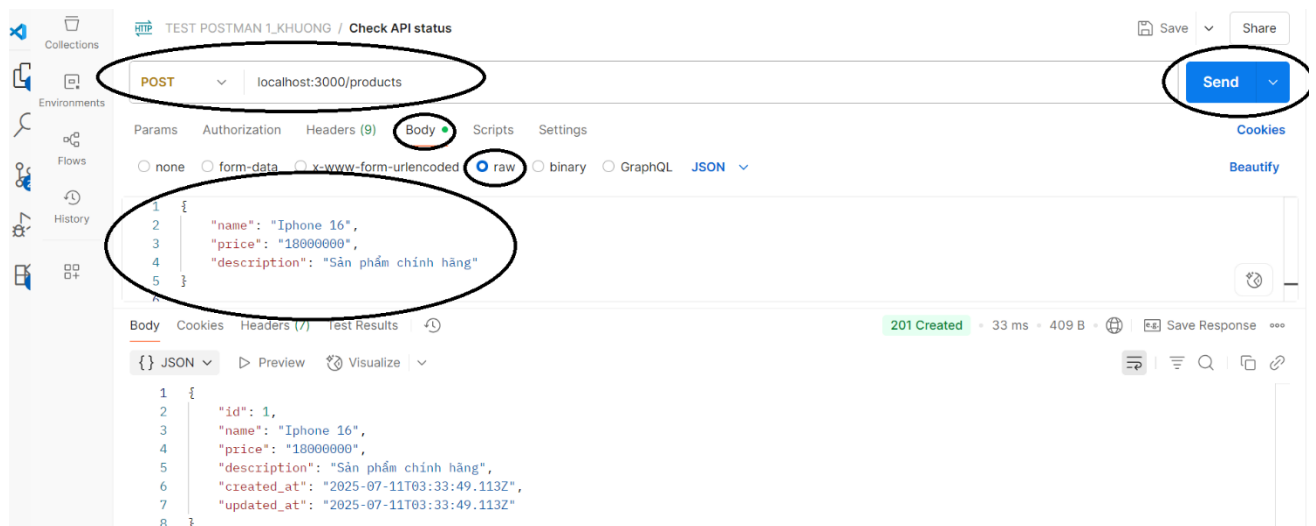
src > modules > products > TS products.controller.ts > ...
1  import { Body, Controller, Delete, Get, Param, Patch, Post } from '@nestjs/common';
2  import { ProductsService } from '../products.service';
3
4  @Controller('products')
5  export class ProductsController {
6      constructor(private readonly productsService: ProductsService) { }
7      @Post()
8      create(@Body() body: any) {
9          return this.productsService.create(body);
10     }
11     @Get()
12     findAll() {
13         return this.productsService.findAll();
14     }
15     @Get(':id')
16     findOne(@Param('id') id: string) {
17         return this.productsService.findOne(+id);
18     }
19     @Patch(':id')
20     update(@Param('id') id: string, @Body() body: any) {
21         return this.productsService.update(+id, body);
22     }
23     @Delete(':id')
24     delete(@Param('id') id: string) {
25         return this.productsService.delete(+id);
26     }
27 }
    
```

6.7. BIÊN DỊCH VÀ KIỂM TRA CHƯƠNG TRÌNH

- Bây giờ các bạn mở **Terminal** và gõ lệnh biên dịch chương trình: **nest start npm run start:dev**

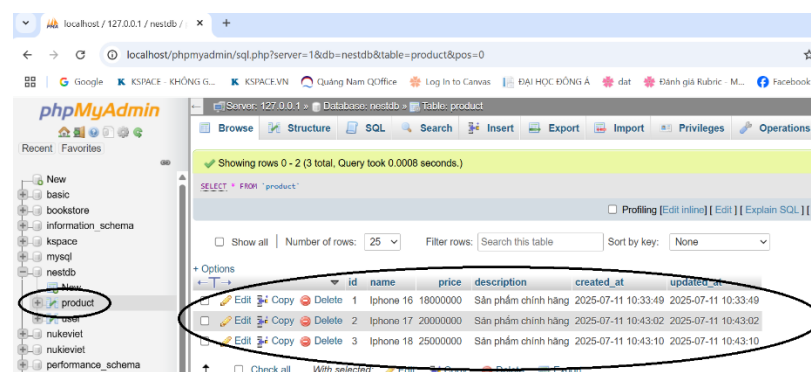


- Kiểm tra hàm **Tạo dữ liệu**: Mở **Postman** và thiết lập cấu hình như bên dưới: Chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/products**, chọn thẻ **Body**, chọn **Raw**, gõ dữ liệu cần nhập vào, chọn **Send**.

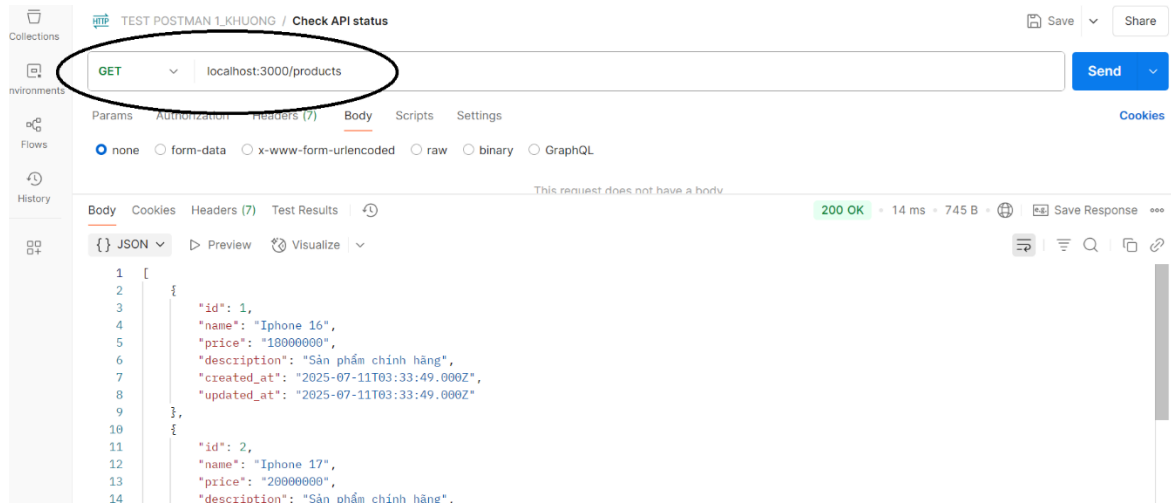


- Các bạn tiếp tục thay đổi dữ liệu trong Raw để thêm 1 vài sản phẩm nữa.

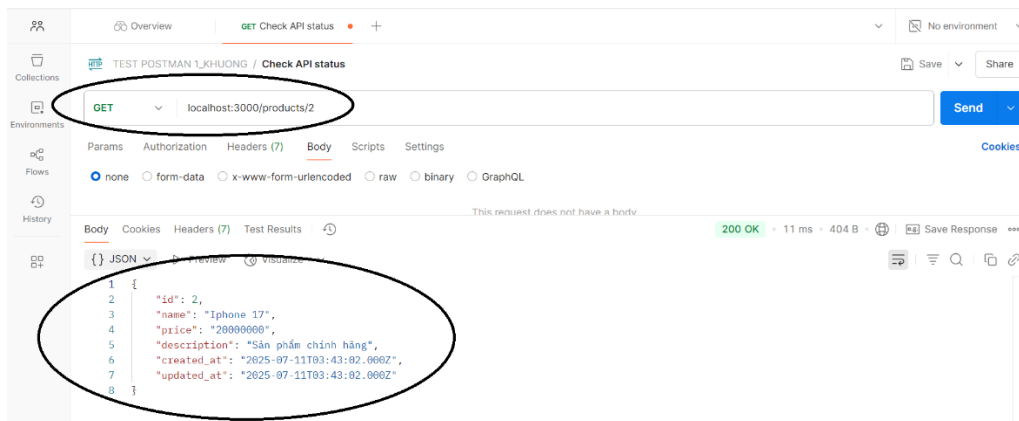
- Mở Cơ sở dữ liệu **MySQL** sẽ thấy đã được thêm vào:



- Kiểm tra phương thức **Hiển thị tất cả**: Thay đổi phương thức là **GET**, gõ đường dẫn: **localhost:3000/products** sẽ thấy hiển thị tất cả dữ liệu:



- Kiểm tra phương thức **Hiển thị dữ liệu theo id**: Vẫn chọn phương thức **GET**, thay đổi đường dẫn: **localhost:3000/products/2** sẽ hiển thị dữ liệu có **id=2**



- Kiểm tra hàm **Cập nhật dữ liệu**: Mở **Postman** và thiết lập cấu hình như bên dưới: Chọn phương thức **PATCH**, gõ đường dẫn: **localhost:3000/products/2**, chọn thẻ **Body**, chọn **Raw**, gõ dữ liệu cần chỉnh sửa cho sản phẩm có **id=2**, chọn **Send**.



- Kiểm tra phương thức **Xóa dữ liệu theo id**: Vẫn chọn phương thức **DELETE**, thay đổi đường dẫn: **localhost:3000/products/2** sẽ xóa dữ liệu có **id=2**

